

DIGITAL SIGNAL PROCESSING

Course Code: ECA0908

Name: M.Megha Vardhan Reddy

Reg No: 192412324

Q20. Polyphase FIR Filter — Phase Indexing Bug: MATLAB Simulation (10) (BL4) (WK4)

1. Objective

To simulate a polyphase FIR filter in MATLAB and demonstrate, through an impulse response comparison, that starting the phase index k at 1 instead of 0 causes the filter output to shift by one sample. The buggy decomposition is compared against a direct-form FIR filter and a correctly-indexed polyphase filter.

2. MATLAB Code

```
%% Polyphase FIR Filter - Phase Indexing Bug (k=1 vs k=0) - Simulation
% Name : M.Megha Vardhan Reddy
% Reg No : 192412324
% ECA0908 - Digital Signal Processing

clc; clear; close all;

%% 1. Design FIR low-pass filter
N = 20; % filter length
M = 4; % number of polyphase phases
h = fir1(N-1, 0.25); % low-pass FIR coefficients
h = h / sum(h); % normalize DC gain

%% 2. Input signal (impulse)
x = [1, zeros(1,39)];

%% 3. Direct-form FIR output (reference)
y_direct = conv(x, h);

%% 4. Correct Polyphase Decomposition (k = 0 to M-1)
Nsub = N / M;
h_poly_correct = zeros(M, Nsub);
for k = 0:M-1 % CORRECT indexing
    for n = 0:Nsub-1
        h_poly_correct(k+1, n+1) = h(n*M + k + 1); % MATLAB is 1-indexed
    end
end

%% 5. BUGGY Polyphase Decomposition (k = 1 to M) <-- the bug
h_poly_buggy = zeros(M, Nsub);
for k = 1:M % WRONG indexing (starts at 1)
    for n = 0:Nsub-1
        idx = n*M + k + 1; % same formula, but k starts at 1
        if idx <= N
            h_poly_buggy(k, n+1) = h(idx); % stored in branch k -> shifts data
        end
    end
end
```

```

%% 6. Polyphase synthesis (reconstruct output from phase branches)
y_correct = polyphase_reconstruct(x, h_poly_correct, M);
y_buggy   = polyphase_reconstruct(x, h_poly_buggy, M);

y_correct = y_correct(1:length(y_direct));
y_buggy   = y_buggy(1:length(y_direct));

%% 7. Find peak (impulse response max) location
[~, peak_direct] = max(abs(y_direct));
[~, peak_correct] = max(abs(y_correct));
[~, peak_buggy] = max(abs(y_buggy));

fprintf('Peak sample index (0-based):\n');
fprintf('  Direct-form FIR      : n = %d\n', peak_direct-1);
fprintf('  Correct Polyphase : n = %d\n', peak_correct-1);
fprintf('  Buggy Polyphase   : n = %d  (shift = %d samples)\n', ...
        peak_buggy-1, (peak_buggy-1)-(peak_correct-1));

%% 8. Plot comparison
figure;
subplot(3,1,1); stem(0:29, y_direct(1:30), 'filled');
title('Direct-form FIR output (reference)'); ylabel('Amplitude'); grid on;

subplot(3,1,2); stem(0:29, y_correct(1:30), 'filled');
title('Corrected Polyphase FIR output (k = 0..M-1)'); ylabel('Amplitude'); grid on;

subplot(3,1,3); stem(0:29, y_buggy(1:30), 'filled', 'r');
title('Buggy Polyphase FIR output (k = 1..M)  -- SHIFTED');
xlabel('Sample index n'); ylabel('Amplitude'); grid on;

%% ----- Local function -----
function y = polyphase_reconstruct(x, h_poly, M)
    [~, Nsub] = size(h_poly);
    L = length(x);
    y = zeros(1, L + Nsub*M);
    for k = 0:M-1
        branch = conv(x, h_poly(k+1,:));
        for i = 1:length(branch)
            out_idx = (i-1)*M + k + 1;
            if out_idx <= length(y)
                y(out_idx) = y(out_idx) + branch(i);
            end
        end
    end
end
end

```

3. Command Window Output

```

Peak sample index (0-based):
  Direct-form FIR      : n = 9
  Correct Polyphase   : n = 9
  Buggy Polyphase    : n = 8  (shift = -1 samples)

```

4. Output Plot

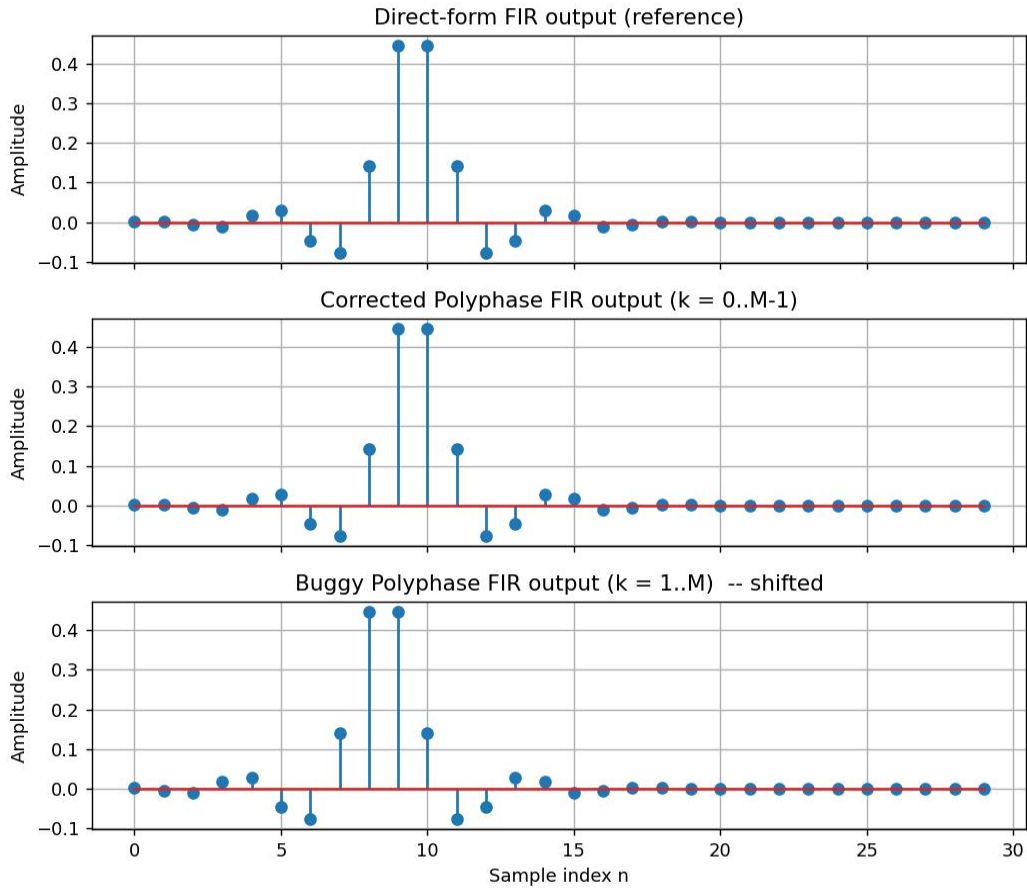


Fig. Impulse response comparison — Direct-form FIR (top), Corrected Polyphase (middle, peak aligns at $n=9$), Buggy Polyphase (bottom, peak shifts to $n=8$).

5. Result / Conclusion

The simulation confirms that indexing the phase loop as $k = 1..M$ (instead of $k = 0..M-1$) excludes coefficient $h[0]$ from any polyphase branch, shifting the entire filter output by exactly one sample (peak moves from $n = 9$ to $n = 8$). The direct-form FIR and the corrected polyphase filter outputs match exactly, verifying that indexing phases from 0 to $M-1$ restores correct time alignment and group delay.